

Date:
Student Name:
Student ID:

AI Workshop #2 Part B

AI with Python - Data Preparation

2.5. Introduction

In the previous workshop, we have already studied supervised as well as unsupervised machine learning algorithms.

These algorithms require formatted data to start the training process. We must prepare or format data in a certain way so that it can be supplied as an input to ML algorithms.

This workshop will focus on the data preparation for machine learning algorithms.

2.6. Preprocessing the Data

In our daily life, we deal with lots of data but this data is in raw form. To provide the data as the input of machine learning algorithms, we need to convert it into a meaningful data.

That is where data preprocessing comes into picture.

In other simple words, we can say that before providing the data to the machine learning algorithms we need to preprocess the data.

2.7. Data Preprocessing Steps

Follow these steps to preprocess the data in Python:

Step 1: Importing the useful packages

If we are using Python then this would be the first step for converting the data into a certain format, i.e., preprocessing. It can be done as follows:

In [1]:

```
import numpy as np
from sklearn import preprocessing
```

Here we have used the following two packages:



- **NumPy**: Basically NumPy is a general purpose array-processing package designed to efficiently manipulate large multi-dimensional arrays of arbitrary records without sacrificing too much speed for small multi-dimensional arrays.
- **Sklearn.preprocessing**: This package provides many common utility functions and transformer classes to change raw feature vectors into a representation that is more suitable for machine learning algorithms.

Step 2: Defining sample data

After importing the packages, we need to define some sample data so that we can apply preprocessing techniques on that data.

For example, we now define the following sample data:

In [2]:

```
input_data = np.array([[2.1, -1.9, 5.5],
                        [-1.5, 2.4, 3.5],
                        [0.5, -7.9, 5.6],
                        [5.9, 2.3, -5.8]])
```

To display the array, use standard print() function:

In [3]:

```
print("Array Input_data is:\n", input_data)
```

```
Array Input_data is:
[[ 2.1 -1.9  5.5]
 [-1.5  2.4  3.5]
 [ 0.5 -7.9  5.6]
 [ 5.9  2.3 -5.8]]
```

Step3: Applying preprocessing technique

In this step, we need to apply any of the preprocessing techniques.

The following section describes the data preprocessing techniques.

2.8 Techniques for Data Preprocessing

The techniques for data preprocessing are described below:

Binarization

This is the preprocessing technique which is used when we need to convert our numerical values into Boolean values. We can use an inbuilt method to binarize the input data say by using 0.5 as the threshold value in the following way:

In [4]:

```
data_binarized = preprocessing.Binarizer(threshold=0.5).transform(input_data)
```

Check the binarized data array:

In [5]:

```
print("\nBinarized data:\n", data_binarized)
```

```
Binarized data:
[[1.  0.  1.]
 [0.  1.  1.]
 [0.  0.  1.]
 [1.  1.  0.]]
```



Note: Binarization is a simple but important data-processing procedure, especially in neural network training. In many neural networks/Deep networks problem, we only need the binary data as input vectors instead of the real number, e.g. car plate number recognition, signature recognition, etc. In these case, binary information of 2D pixel array are used as input vectors.

In our example, after running the above code we will get the following output, all the values above 0.5(threshold value) would be converted to 1 and all the values below 0.5 would be converted to 0.

Mean Removal

It is another very common preprocessing technique that is used in machine learning.

Basically it is used to eliminate the mean from feature vector so that every feature is centered on zero. We can also remove the bias from the features in the feature vector.

For applying mean removal preprocessing technique on the sample data, we can write the Python code shown below. The code will display the Mean and Standard deviation of the input data:

In [6]:

```
print("Mean =", input_data.mean(axis=0))
print("Std deviation = ", input_data.std(axis=0))
```

```
Mean = [ 1.75 -1.275  2.2 ]
Std deviation = [2.71431391 4.20022321 4.69414529]
```

Now, the code below will remove the **Mean** and **Standard deviation** of the input data:

In [7]:

```
data_scaled = preprocessing.scale(input_data)
print("Mean =", data_scaled.mean(axis=0))
print("Std deviation =", data_scaled.std(axis=0))
```

```
Mean = [1.11022302e-16 0.00000000e+00 0.00000000e+00]
Std deviation = [1. 1. 1.]
```

Scaling

It is another data preprocessing technique that is used to scale the feature vectors.

Scaling of feature vectors is needed because the values of every feature can vary between many random values.

In other words we can say that scaling is important because we do not want any feature to be synthetically large or small.

With the help of the following Python code, we can do the scaling of our input data, i.e., feature vector:

Min max Scaling

In [8]:

```
data_scaler_minmax = preprocessing.MinMaxScaler(feature_range=(0,1))
data_scaled_minmax = data_scaler_minmax.fit_transform(input_data)
print ("\nMin max scaled data:\n", data_scaled_minmax)
```

```
Min max scaled data:
[[0.48648649 0.58252427 0.99122807]
 [0.         1.         0.81578947]
 [0.27027027 0.         1.         ]
 [1.         0.99029126 0.         ]]
```



Workshop 2.1 - Data Scaling

Scaling is an important data-preprocessing technique in many AI problem, especially for the data processing for ANN/Deep Network.

Using weather forecast as example, discuss how Scaling Method is used for weather prediction with various input vectors such as Temperature, Air Pressure, Relative Humidity, Wind Speed and Director, etc.

Workshop 2.1 Answer:

Normalization

Normalization is another data preprocessing technique that is used to modify the feature vectors.

Such kind of modification is necessary to measure the feature vectors on a common scale.

Followings are two types of normalization which can be used in machine learning:

L1 Normalization

It is also referred to as **Least Absolute Deviations**.

This kind of normalization modifies the values so that the sum of the absolute values is always up to 1 in each row.

It can be implemented on the input data with the help of the following Python code:

In [9]:

```
# Normalize data
data_normalized_l1 = preprocessing.normalize(input_data, norm='l1')
```

In [10]:

```
print("\nL1 normalized data:\n", data_normalized_l1)
```

```
L1 normalized data:
[[ 0.22105263 -0.2          0.57894737]
 [-0.2027027   0.32432432  0.47297297]
 [ 0.03571429 -0.56428571  0.4        ]
 [ 0.42142857  0.16428571 -0.41428571]]
```

L2 Normalization

It is also referred to as **least squares normalization**.

This kind of normalization modifies the values so that the sum of the squares is always up to 1 in each row.

It can be implemented on the input data with the help of the following Python code:

In [11]:

```
# Normalize data
data_normalized_l2 = preprocessing.normalize(input_data, norm='l2')
```

In [12]:

```
print("\nL2 normalized data:\n", data_normalized_l2)
```

```
L2 normalized data:
[[ 0.33946114 -0.30713151  0.88906489]
 [-0.33325106  0.53320169  0.7775858 ]
 [ 0.05156558 -0.81473612  0.57753446]
 [ 0.68706914  0.26784051 -0.6754239 ]]
```



Workshop 2.2 - Normalization Techniques

Compare standard, L1 and L2 normalization, please discuss what are there main difference in terms of data preprocessing. And discuss (with examples) what are the problem domains (data types) that best fit for each normalization scheme.

Workshop 3.2 Answer:

2.9 Labeling the Data

We already know that data in a certain format is necessary for machine learning algorithms.

Another important requirement is that the data must be labelled properly before sending it as the input of machine learning algorithms.

For example, if we talk about classification, there are lot of labels on the data. Those labels are in the form of words, numbers, etc.

Functions related to machine learning in sklearn expect that the data must have number labels.

Hence, if the data is in other form then it must be converted to numbers. This process of transforming the word labels into numerical form is called **label encoding**.

Label encoding steps

Follow these steps for encoding the data labels in Python:

Step1: Importing the useful packages

If we are using Python then this would be first step for converting the data into certain format, i.e., preprocessing. It can be done as follows:

In [13]:

```
import numpy as np
from sklearn import preprocessing
```

Step 2: Defining sample labels

After importing the packages, we need to define some sample labels so that we can create and train the label encoder.

We will now define the following sample labels:

In [14]:

```
# Sample input labels
input_labels = ['red', 'black', 'red', 'green', 'black', 'yellow', 'white']
```

In [15]:

```
input_labels
```

Out[15]:

```
['red', 'black', 'red', 'green', 'black', 'yellow', 'white']
```

Step 3: Creating & training of label encoder object

In this step, we need to create the label encoder and train it.

The following Python code will help in doing this:

In [16]:

```
# Creating the label encoder
encoder = preprocessing.LabelEncoder()
encoder.fit(input_labels)
```

Out[16]:

```
▼ LabelEncoder
LabelEncoder()
```

Step4: Checking the performance by encoding random ordered list

This step can be used to check the performance by encoding the random ordered list.

Following Python code can be written to do the same:

In [17]:

```
# encoding a set of labels
test_labels = ['green', 'red', 'black']
encoded_values = encoder.transform(test_labels)
```

Check the test_labels:

In [18]:

```
print("\nLabels =", test_labels)
```

```
Labels = ['green', 'red', 'black']
```

Now, we can get the list of encoded values i.e. word labels converted to numbers as follows:

In [19]:

```
print("Encoded values =", list(encoded_values))
```

```
Encoded values = [1, 2, 0]
```

Step 5: Checking the performance by decoding a random set of numbers:

This step can be used to check the performance by decoding the random set of numbers.

Following Python code can be written to do the same:

In [20]:

```
# decoding a set of values
encoded_values = [3, 0, 4, 1]
decoded_list = encoder.inverse_transform(encoded_values)
```

Check the encoded values:

In [21]:

```
print("\nEncoded values =", encoded_values)
```

Encoded values = [3, 0, 4, 1]

Now, decoded values would get printed as follows:

In [22]:

```
print("\nDecoded labels =", list(decoded_list))
```

Decoded labels = ['white', 'black', 'yellow', 'green']



Workshop 2.3 - Data Labeling

In many AI classification problem, we using come across input (vectors) which are NON number in nature.

1. Please give THREE examples of such AI problem and describe HOW it can be handled by using **Data Labeling** method.
2. Discuss the pros and cons of such method.

Workshop 2.3 Answer:

In [23]:

```
## Workshop 2.3 Coding Solution ##
```

2.10 Labeled v/s Unlabeled Data

Unlabeled data mainly consists of the samples of natural or human-created object that can easily be obtained from the world. They include, audio, video, photos, news articles, etc.

On the other hand, labeled data takes a set of unlabeled data and augments each piece of that unlabeled data with some tag or label or class that is meaningful. For example, if we have a photo then the label can be put based on the content of the photo, i.e., it is photo of a boy or girl or animal or anything else. Labeling the data needs human expertise or judgment about a given piece of unlabeled data.

There are many scenarios where unlabeled data is plentiful and easily obtained but labeled data often requires a human/expert to annotate. Semi-supervised learning attempts to combine labeled and unlabeled data to build better models.



Workshop 2.4

Data preprocessing is a critical procedure in many real world machine learning and AI problem. Using weather forecast as example, various data preprocessing such as data normalization, scaling and labeling are needed before the time-series weather information can be used for network training and testing.

Use the time series weather data of Seattle (weather.csv) provided in this workshop as the time-series raw data for data preprocessing:

1. Describe and explain the nature of data in each attribute of the time series records.
2. Discuss what kind of data preprocessing methods are needed for each attribute.
3. How about missing record and incorrect data, how can we fix such problems.
4. Write Python program to implement the data processing method.

Hint: The **normal** range and condition of each weather attribute are:

1. Air Pressure 900 - 1200
2. Precipitation 0 - 300
3. Temperature -50 - 50
4. Max >= Min Temp
5. Wind Speed (Grade) 0 - 10
6. Wind Direction 0 - 360

Workshop 2.4 Answer:

In [24]:

Workshop 2.4 Coding Solution

In []: